

Formal Verification of BayesianNetworks.jl

Simon Frost

2026-09-07

Contents

BayesianNetworksProofs	1
What is formalised	2
SPEC §61 propositions	2
BayesianNetworksProofs.Schema.Desc	3
ACSets.jl schema JSON	4
BayesianNetworksProofs.Schema.BayesNet	5
Theorems	6
BayesianNetworksProofs.Finite.BayesNet	6
BayesianNetworksProofs.Finite.Evaluation	8
Fibres and marginalisation	8
Partial joints along a list of variables	9
Proposition 1	10
Marginalising the downstream variables out (Proposition 3, closed-world shadow)	11
BayesianNetworksProofs.Finite.Intervention	12
BayesianNetworksProofs.Finite.Tensor	14
BayesianNetworksProofs.Finite.Open	16
Composition without quotients	17
Semantics (Proposition 3)	17
Semantics	21
BayesianNetworksProofs.Markov.Basic	24
BayesianNetworksProofs.Finite.Probability	25
Roadmap (contains sorry)	26
Proposition 3 — the interface form	26

BayesianNetworksProofs

```
import Mathlib.CategoryTheory.MarkovCategory.Basic
```

Lean 4 / Mathlib formalisation accompanying BayesianNetworks.jl (toolchain leanprover/lean4:v4.30.0, Mathlib tag v4.30.0; ADR 0005). This document is generated from the Lean sources by [mdgen](#): the prose is the module docstrings and the code blocks are the verbatim, machine-checked sources. Every declaration outside the final “Roadmap” section is built by `lake build --wfail` and its axioms are printed by `Audit.lean` (only `propext`, `Classical.choice`, `Quot.sound`).

What is formalised

`BayesianNetworks.jl` represents a Bayesian network as an ACSet on the schema `SchBayesNet` (SPEC §8) whose semantics is a finite stochastic state $p_G : I \rightarrow \otimes_v X_v$ computed by `joint_distribution` (SPEC §5, §10), with hard interventions `do_intervention` (SPEC §21–§22) and the tensor otimes of open networks (SPEC §13). Three layers are formalised:

- **Schema layer** (`Schema/`, Mathlib-free): the ACSet schemas as Lean terms, their well-formedness, and the JSON writer behind `lake exe emit_schema`. The Julia test compares `generate_json_acset_schema(SchBayesNet)` with the committed schemas/`*.schema.json`, so the The Lean terms and the Julia @present schemas are two independent definitions that `emit_schema` --check and a Julia test check to agree; neither is generated from the other.
- **Finite model** (`Finite/`): `FinBayesNet` (finite variables and mechanisms, finite state spaces, target, parents), kernel families over an arbitrary commutative semiring R (instantiated at $R \geq 0$ in `Finite/Probability.lean`), Propositions 1, 2, 3 and 4 of SPEC §61, and the open-network closure theorem of SPEC §13 (`Finite/Open.lean`).
- **Abstract layer** (`Markov/`): the generic consequences of Mathlib’s `MarkovCategory / CopyDiscardCategory` axioms that the Julia tests check on the finite-stochastic instance.

Part	Module	Content
1	<code>Schema/Desc.lean</code>	<code>SchemaDesc</code> , decidable WF, Sub, NoOutgoing, extend, toACSetsJson (ACSets.jl’s <code>generate_json_acset_schema</code> format).
2	<code>Schema/BayesNet.lean</code>	<code>schVariableSpace</code> , <code>schBayesNet</code> , <code>schInfluenceDiagram</code> and their decided properties, including the <code>OpenACSetTypes</code> precondition.
3	<code>Finite/BayesNet.lean</code>	<code>FinBayesNet</code> , <code>Assignment</code> , <code>Kernel R</code> , <code>LocalOn</code> , <code>Local</code> , <code>Normalised</code> , <code>Closed</code> , <code>TopoOrder</code> , <code>joint</code> .
4	<code>Finite/Evaluation.lean</code>	Proposition 1: <code>sum_joint_eq_one</code> (1a) and <code>evalSeq_eq_joint</code> (1b), via fibres and marginalisation.
5	<code>Finite/Intervention.lean</code>	Proposition 4: <code>cut</code> , <code>intervene</code> , <code>joint_intervene</code> , <code>sum_joint_intervene_eq_one</code> .
6	<code>Finite/Tensor.lean</code>	Proposition 2: <code>tensor</code> , <code>tensorKernel</code> , <code>joint_tensor</code> , <code>TopoOrder.tensor</code> .
7	<code>Finite/Open.lean</code>	<code>OpenFinBayesNet</code> (the typed-interface rule), <code>Composable</code> , <code>compose</code> — the open-network closure theorem — and Proposition 3: <code>joint_compose</code> , <code>marg_joint_compose</code> , <code>marg_joint_compose_split</code> .
8	<code>Markov/Basic.lean</code>	<code>discard_natural</code> , <code>state_discard</code> , <code>deterministic_comp</code> , <code>deterministic_copy</code> for Mathlib’s abstract classes.
9	<code>Finite/Probability.lean</code>	Propositions 1 and 4 at $R := \mathbb{R} \geq 0$.
—	<code>Roadmap.lean</code>	Proposition 3 in interface form (<code>osem_compose</code> , sorry); not in the default target.

SPEC §61 propositions

SPEC §61	Lean	Julia (<code>BayesianNetworks.jl</code>)
Prop 1 — BN evaluation, $\llbracket G \rrbracket(x) = \prod_v \kappa_v(x_v \mid x_{pa(v)})$	<code>sum_joint_eq_one</code> (the product is a distribution), <code>evalSeq_eq_joint</code> (the sequential evaluator computes it)	<code>joint_distribution(bn)</code> , <code>validate(bn; closed=true)</code> , <code>topological_order</code>
Prop 2 — tensor compositionality, $\llbracket A \otimes B \rrbracket = \llbracket A \rrbracket \otimes \llbracket B \rrbracket$	<code>joint_tensor</code> , <code>closed_tensor</code> , <code>normalised_tensor</code> , <code>local_tensor</code> , <code>TopoOrder.tensor</code>	<code>otimes(A, B)</code> of <code>OpenBayesNets</code>

SPEC §61	Lean	Julia (BayesianNetworks.jl)
Prop 3 — sequential compositionality, $\llbracket B \circ A \rrbracket = \llbracket B \rrbracket \circ \llbracket A \rrbracket$	<code>marg_joint_upstream(closed-world shadow), joint_compose, marg_joint_compose, marg_joint_compose_split</code> (open networks); Roadmap.osem_compose restates the last one in terms of the interface sets and is still unproved	<code>compose(A, B), glue(A, B; along)</code> of OpenBayesNets
SPEC §13 revision note — pushout composition preserves “at most one mechanism per variable”	<code>composeNet_target_injective, compose_input_exogenous, compose_exogenous_input, composeTopo, compose</code>	<code>Open(bn; inputs, outputs), validate(::OpenBayesNet), compose, glue</code>
Prop 4 — hard intervention, truncated factorisation of $\text{do}(X = x)$	<code>joint_intervene, normalised_intervene, local_cut, sum_joint_intervene_eq_one</code>	<code>do_intervention(bn, :X $\Rightarrow$ x)</code>
Props 5–7 — influence diagrams	<code>InfluenceDiagrams.jl/proofs</code> (depends on this project by path)	<code>InfluenceDiagrams.jl</code>

The Markov-category laws of the kernel calculus itself (`compose`, `otimes`, `mcop`, `delete`) are proved in `MarkovCategories.jl/proofs`.

```
namespace BayesianNetworksProofs
```

```
-- Smoke lemma so that an empty project builds and the axiom audit has something to print. -/
theorem smoke : (1 : Nat) + 1 = 2 := rfl
```

```
end BayesianNetworksProofs
```

BayesianNetworksProofs.Schema.Desc

```
import Lean.Data.Json
```

Schema descriptions — a small, Mathlib-free data type describing the *presentation* of an ACSet schema (objects, homs, attribute types, attributes), together with the decidable well-formedness / inclusion / “no outgoing arrow” predicates that the Julia layer relies on and the JSON writer that reproduces ACSets.jl’s `generate_json_acset_schema` format.

The Lean terms here and the `@present` schemas in `BayesianNetworks.jl` are two independent hand-written definitions, checked to agree rather than generated from one another: `lake exe emit_schema` writes `proofs/schemas/*.schema.json` from the terms in `BayesianNetworksProofs.Schema.BayesNet`, and a Julia test asserts that the `@present` schema matches (ADR 0005).

This module deliberately imports only `Lean.Data.Json` so that the `emit_schema` executable links against Lean core alone (no Mathlib object code is required).

```
namespace BayesianNetworksProofs
```

```
open Lean (Json ToJson FromJson)
```

```
-- A generator of arrow shape: a hom 'name : dom  $\rightarrow$  cod' between objects, or an attribute 'name : dom  $\rightarrow$  cod' from an object to an attribute type. -/
```

```
structure Arrow where
  name : String
  dom : String
  cod : String
  deriving ToJson, FromJson, DecidableEq, Repr
```

```
-- A presentation of an ACSet schema: object generators, hom generators, attribute-type generators and attribute generators. Equations are not modelled (the schemas of 'BayesianNetworks.jl' have none). -/
```

```
structure SchemaDesc where
```

```

obs : List String
homs : List Arrow
attrtypes : List String
attrs : List Arrow
deriving ToJson, FromJson, DecidableEq, Repr

namespace SchemaDesc

/-- All generator names of a schema, in the order 'obs, homs, attrtypes, attrs'. -/
def names (S : SchemaDesc) : List String :=
  S.obs ++ S.homs.map Arrow.name ++ S.attrtypes ++ S.attrs.map Arrow.name

/-- Well-formedness: every generator name is distinct; every hom has an object as domain and
codomain; every attribute has an object as domain and an attribute type as codomain. -/
def WF (S : SchemaDesc) : Prop :=
  S.names.Nodup ∧
  (∀ h ∈ S.homs, h.dom ∈ S.obs ∧ h.cod ∈ S.obs) ∧
  (∀ a ∈ S.attrs, a.dom ∈ S.obs ∧ a.cod ∈ S.attrtypes)

instance (S : SchemaDesc) : Decidable S.WF := by unfold WF; infer_instance

/-- 'Sub S₀ S': every generator of 'S₀' is a generator of 'S' (generator-wise inclusion, the
shape of a Catlab '@present Sch <: Sch₀'). -/
def Sub (S₀ S : SchemaDesc) : Prop :=
  (∀ x ∈ S₀.obs, x ∈ S.obs) ∧ (∀ h ∈ S₀.homs, h ∈ S.homs) ∧
  (∀ x ∈ S₀.attrtypes, x ∈ S.attrtypes) ∧ (∀ a ∈ S₀.attrs, a ∈ S.attrs)

instance (S₀ S : SchemaDesc) : Decidable (Sub S₀ S) := by unfold Sub; infer_instance

/-- 'NoOutgoing S₀ S': no hom or attribute of 'S' whose domain is an object of 'S₀' has a
codomain outside 'S₀'. This is the precondition of Catlab's multi-object 'OpenACSetTypes':
the sub-schema 'S₀' can then serve as the interface (foot) type of open 'S'-ACSets. -/
def NoOutgoing (S₀ S : SchemaDesc) : Prop :=
  (∀ h ∈ S.homs, h.dom ∈ S₀.obs → h.cod ∈ S₀.obs) ∧
  (∀ a ∈ S.attrs, a.dom ∈ S₀.obs → a.cod ∈ S₀.attrtypes)

instance (S₀ S : SchemaDesc) : Decidable (NoOutgoing S₀ S) := by unfold NoOutgoing; infer_instance

/-- Extend a schema by appending generators (the Lean counterpart of '@present Sch <: Sch₀'). -/
def extend (S : SchemaDesc) (obs : List String := []) (homs : List Arrow := [])
  (attrtypes : List String := []) (attrs : List Arrow := []) : SchemaDesc where
  obs := S.obs ++ obs
  homs := S.homs ++ homs
  attrtypes := S.attrtypes ++ attrtypes
  attrs := S.attrs ++ attrs

/-- An extension always contains the schema it extends. -/
theorem sub_extend (S : SchemaDesc) (obs homs attrtypes attrs) :
  Sub S (S.extend obs homs attrtypes attrs) := by
  refine {?_, ?_, ?_, ?_} <=> intro x hx <=> simp [extend, hx]

```

ACSets.jl schema JSON

generate_json_acset_schema in ACSets/src/serialization/JSONACSets.jl emits

```

{ "version": {"ACSetSchema": "0.0.1", "ACSets": <pkg version>},
  "Ob": [{"name": ..}], "Hom": [{"name", "dom", "codom"}],
  "AttrType": [{"name"}], "Attr": [{"name", "dom", "codom"}],
  "equations": [] }

```

(codomains are spelled codom; Ob/AttrType entries are objects with a single name). JSON objects are unordered; Lean's Json.obj stores keys in an RB-map, so Json.pretty prints keys in a fixed (reverse-alphabetical) order. Consumers must compare structurally.

```

/-- The 'ACSetSchema' format version written by ACSets.jl. -/
def acsetSchemaVersion : String := "0.0.1"

```

```

/-- The ACSets.jl package version recorded in emitted files (informational; Julia compares
modulo 'version'). -/
def acsetsPkgVersion : String := "0.2.29"

private def arrowJson (a : Arrow) : Json :=
  Json.mkObj [{"name", a.name}, {"dom", a.dom}, {"codom", a.cod}]

private def namedJson (x : String) : Json := Json.mkObj [{"name", x}]

/-- Render a schema description in the ACSets.jl 'generate_json_acset_schema' format. -/
def toACSetsJson (S : SchemaDesc) : Json :=
  Json.mkObj
    [ ("version", Json.mkObj [{"ACSetSchema", acsetSchemaVersion}, {"ACSets", acsetsPkgVersion}])
    , ("Ob", Json.arr (S.obs.map namedJson).toArray)
    , ("Hom", Json.arr (S.homs.map arrowJson).toArray)
    , ("AttrType", Json.arr (S.attrtypes.map namedJson).toArray)
    , ("Attr", Json.arr (S.attrs.map arrowJson).toArray)
    , ("equations", Json.arr #[] )

/-- Pretty-printed, deterministic text of 'toACSetsJson' (trailing newline included). -/
def toACSetsJsonString (S : SchemaDesc) : String :=
  (S.toACSetsJson.pretty 100) ++ "\n"

end SchemaDesc

end BayesianNetworksProofs

```

BayesianNetworksProofs.Schema.BayesNet

```
import BayesianNetworksProofs.Schema.Desc
```

The three ACSet schemas of the ecosystem, as SchemaDesc terms:

- schVariableSpace — the interface sub-schema (variables and their states);
- schBayesNet — adds mechanisms and their ordered inputs (SPEC §8);
- schInfluenceDiagram — adds decisions, information arcs, utilities and decision precedence (SPEC §24).

The theorems (all by decide) are what the Julia layer assumes: each schema is well formed, each extension contains its base, and schVariableSpace has *no outgoing arrows* in schBayesNet / schInfluenceDiagram, which is the precondition of Catlab's multi-object OpenACSetTypes(BayesNet, VariableSpace).

```

namespace BayesianNetworksProofs

open SchemaDesc

/-- 'SchVariableSpace': objects 'Variable', 'State'; hom 'state_variable'; attribute types
'Label', 'Position', 'Ref'; attributes naming variables and states, positioning states and
referencing the semantic state space. -/
def schVariableSpace : SchemaDesc where
  obs := ["Variable", "State"]
  homs := [{"state_variable", "State", "Variable"}]
  attrtypes := ["Label", "Position", "Ref"]
  attrs :=
    [ {<"variable_name", "Variable", "Label">
      , {<"space_ref", "Variable", "Ref">
      , {<"state_name", "State", "Label">
      , {<"state_position", "State", "Position"> } ]

/-- 'SchBayesNet <: SchVariableSpace': mechanisms with a 'target' variable and ordered inputs. -/
def schBayesNet : SchemaDesc :=
  schVariableSpace.extend
    (obs := ["Mechanism", "Input"])
    (homs :=

```

```

    [ <"target", "Mechanism", "Variable">
      , <"input_mechanism", "Input", "Mechanism">
      , <"input_variable", "Input", "Variable"> ] )
  (attrs :=
    [ <"mechanism_name", "Mechanism", "Label">
      , <"kernel_ref", "Mechanism", "Ref">
      , <"input_position", "Input", "Position"> ] )

/-- 'SchInfluenceDiagram <: SchBayesNet': decisions, information inputs, utilities with their
inputs, and decision precedence. -/
def schInfluenceDiagram : SchemaDesc :=
  schBayesNet.extend
    (obs := ["Decision", "InformationInput", "Utility", "UtilityInput", "DecisionPrecedence"])
    (homs :=
      [ <"decision_variable", "Decision", "Variable">
        , <"information_decision", "InformationInput", "Decision">
        , <"information_variable", "InformationInput", "Variable">
        , <"utility_node", "UtilityInput", "Utility">
        , <"utility_variable", "UtilityInput", "Variable">
        , <"earlier", "DecisionPrecedence", "Decision">
        , <"later", "DecisionPrecedence", "Decision"> ] )
    (attrs :=
      [ <"decision_name", "Decision", "Label">
        , <"utility_name", "Utility", "Label">
        , <"utility_ref", "Utility", "Ref">
        , <"information_position", "InformationInput", "Position">
        , <"utility_position", "UtilityInput", "Position"> ] )

```

Theorems

```

theorem schVariableSpace_wf : schVariableSpace.WF := by decide

theorem schBayesNet_wf : schBayesNet.WF := by decide

theorem schInfluenceDiagram_wf : schInfluenceDiagram.WF := by decide

theorem sub_variableSpace_bayesNet : Sub schVariableSpace schBayesNet := by decide

/-- No hom or attribute of 'SchBayesNet' leaves '{Variable, State}': 'VariableSpace' is a valid
interface type for 'OpenACSetTypes(BayesNet, VariableSpace)'. -/
theorem noOutgoing_variableSpace_bayesNet : NoOutgoing schVariableSpace schBayesNet := by decide

theorem sub_bayesNet_influenceDiagram : Sub schBayesNet schInfluenceDiagram := by decide

theorem noOutgoing_variableSpace_influenceDiagram :
  NoOutgoing schVariableSpace schInfluenceDiagram := by decide

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.BayesNet

```

import Mathlib.Data.Fintype.Pi
import Mathlib.Algebra.BigOperators.Group.Finset.Basic

```

Concrete finite Bayesian networks — the semantic model behind `BayesianNetworks.jl`'s finite stochastic evaluator (SPEC §5, §8, §10, §14).

A `FinBayesNet` is a finite type V of variables, a finite type M of mechanisms, a finite non-empty state space $states$ v for each variable, a target variable for each mechanism and a finite set of parents for each mechanism. This is the shape of the `SchBayesNet ACSet (Mechanism --target--> Variable, Input rows collected into parents)` with the attribute data erased; kernels live outside the structure, exactly as `kernel_ref` points outside the `ACSet` in Julia.

Kernels are valued in an arbitrary commutative semiring R (instantiate with $\mathbb{R}_{\geq 0}$ or R): $\kappa \ m \ x \ y$ is the weight the mechanism m assigns to the value y of its target given the assignment x . Two properties matter:

- Local $\kappa \ m \ — \ \kappa \ m \ x$ depends on x only through the parents of m (the mechanism is a kernel $\otimes \text{parents} \rightarrow \text{target}$);
- Normalised $\kappa \ m \ — \ \sum y, \kappa \ m \ x \ y = 1$ (the kernel is a stochastic map).

Closed bn (exactly one mechanism per variable, target bijective) is `validate(bn; closed=true)` and `TopoOrder bn` is a topological order of the derived variable graph (SPEC §11 item 7).

The joint $\kappa \ x = \prod m, \kappa \ m \ x \ (x \ (\text{target } m))$ is Proposition 1's right-hand side.

```
namespace BayesianNetworksProofs

/-- A finite Bayesian-network #shape*: variables, mechanisms, state spaces, targets, parents. -/
structure FinBayesNet where
  /-- Variables. -/
  V : Type
  /-- Mechanisms (one per 'Mechanism' row of the ACSet). -/
  M : Type
  [fintypeV : Fintype V]
  [decV : DecidableEq V]
  [fintypeM : Fintype M]
  [decM : DecidableEq M]
  /-- The finite state space of each variable. -/
  states : V → Type
  [fintypeS : ∀ v, Fintype (states v)]
  [decS : ∀ v, DecidableEq (states v)]
  [nonemptyS : ∀ v, Nonempty (states v)]
  /-- The variable generated by each mechanism ('target : Mechanism → Variable'). -/
  target : M → V
  /-- The input variables of each mechanism (the 'Input' rows of the ACSet). -/
  parents : M → Finset V

attribute [instance] FinBayesNet.fintypeV FinBayesNet.decV FinBayesNet.fintypeM FinBayesNet.decM
  FinBayesNet.fintypeS FinBayesNet.decS FinBayesNet.nonemptyS

namespace FinBayesNet

variable (bn : FinBayesNet)

/-- A joint assignment of a state to every variable. -/
abbrev Assignment := ∀ v, bn.states v

/-- A family of conditional kernels, one per mechanism: ' $\kappa \ m \ x \ y$ ' is the weight of the target
value ' $y$ ' given the assignment ' $x$ '. -/
abbrev Kernel (R : Type) := (m : bn.M) → bn.Assignment → bn.states (bn.target m) → R

variable {bn} {R : Type}

/-- ' $k$ ' depends on the assignment only through the variables in ' $P$ '. -/
def LocalOn (P : Finset bn.V) {v : bn.V} (k : bn.Assignment → bn.states v → R) : Prop :=
  ∀ x x' : bn.Assignment, (∀ p ∈ P, x p = x' p) → k x = k x'

/-- The kernel of ' $m$ ' reads only the parents of ' $m$ '. -/
def Local (κ : bn.Kernel R) (m : bn.M) : Prop := LocalOn (bn.parents m) (κ m)

/-- The kernel of ' $m$ ' is a stochastic map: its weights sum to one for every assignment. -/
def Normalised [AddCommMonoid R] [One R] (κ : bn.Kernel R) (m : bn.M) : Prop :=
  ∀ x, ∑ y, κ m x y = 1

/-- Closed network: every variable has exactly one generating mechanism
('validate(bn; closed=true)'). -/
def Closed (bn : FinBayesNet) : Prop := Function.Bijective bn.target

/-- A topological order of the variables: a duplicate-free, complete list in which every parent
of a mechanism precedes the mechanism's target, and no mechanism reads its own target. -/
```

```

structure TopoOrder (bn : FinBayesNet) where
  order : List bn.V
  nodup : order.Nodup
  complete : ∀ v, v ∈ order
  /-- For 'a' before 'b' in the order, 'b' is not a parent of the mechanism generating 'a'. -/
  parents_before : order.Pairwise (fun a b ⇒ ∀ m, bn.target m = a → b ∉ bn.parents m)
  no_self : ∀ m, bn.target m ∉ bn.parents m

  /-- The joint weight of an assignment: the product of all mechanism kernels evaluated at it
  (Proposition 1, right-hand side). -/
def joint [CommMonoid R] (κ : bn.Kernel R) (x : bn.Assignment) : R :=
  ∏ m, κ m x (x (bn.target m))

end FinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.Evaluation

```

import BayesianNetworksProofs.Finite.BayesNet
import Mathlib.Algebra.BigOperators.Ring.Finset

```

Proposition 1 (BN evaluation) for the concrete finite model.

- `sum_joint_eq_one` (Prop 1a): for a closed network with local, normalised kernels the joint $\prod m, \kappa m x (x (target\ m))$ sums to one over all assignments, i.e. it is a probability distribution — the state $p_G : I \rightarrow \otimes X_v$ of SPEC §5.
- `evalSeq_eq_joint` (Prop 1b): the sequential evaluator that walks a topological order and multiplies in the kernels of the mechanisms generating each variable computes the same product.

The proof of 1a integrates the variables out one at a time along the topological order. To stay free of dependent tuples, “integrating out S ” is expressed with `fibre S x₀`, the finite set of assignments agreeing with a base assignment x_0 outside S ; `marg S F x₀` sums F over that fibre. Adding a variable to S splits the fibre by the value of that variable (`marg_insert`), and locality lets the kernel of the newest variable be pulled out of the sum.

```

namespace BayesianNetworksProofs

namespace FinBayesNet

variable {bn : FinBayesNet} {R : Type}

```

Fibres and marginalisation

```

/-- Assignments agreeing with 'x₀' outside 'S'. -/
def fibre (S : Finset bn.V) (x₀ : bn.Assignment) : Finset bn.Assignment :=
  Finset.univ.filter (fun y ⇒ ∀ v, v ∉ S → y v = x₀ v)

theorem mem_fibre {S : Finset bn.V} {x₀ y : bn.Assignment} :
  y ∈ fibre S x₀ ↔ ∀ v, v ∉ S → y v = x₀ v := by
  simp [fibre]

theorem fibre_empty (x₀ : bn.Assignment) : fibre (∅ : Finset bn.V) x₀ = {x₀} := by
  ext y
  simp [mem_fibre, funext_iff]

theorem fibre_univ (x₀ : bn.Assignment) : fibre (Finset.univ : Finset bn.V) x₀ = Finset.univ := by
  ext y
  simp [mem_fibre]

/-- Splitting the fibre of 'insert v₀ S' by the value at 'v₀'. -/
theorem fibre_insert_filter {S : Finset bn.V} {v₀ : bn.V} (hv : v₀ ∉ S) (x₀ : bn.Assignment)
  (z : bn.states v₀) :

```

```

(fibre (insert v₀ S) x₀).filter (fun y ⇒ y v₀ = z) = fibre S (Function.update x₀ v₀ z) := by
ext y
simp only [Finset.mem_filter, mem_fibre, Finset.mem_insert, not_or]
constructor
• rintro ⟨h, hz⟩ v hvS
  by_cases hvv : v = v₀
  • subst hvv
    simp [hz]
  • rw [Function.update_of_ne hvv]
    exact h v ⟨hvv, hvS⟩
• intro h
  refine ⟨fun v ⟨hvv, hvS⟩ ⇒ ?, ?⟩
  • have := h v hvS
    rwa [Function.update_of_ne hvv] at this
  • have := h v₀ hv
    simp using this

variable [CommSemiring R]

/-- Sum of 'F' over the assignments agreeing with 'x₀' outside 'S'. -/
def marg (S : Finset bn.V) (F : bn.Assignment → R) (x₀ : bn.Assignment) : R :=
  ∑ y ∈ fibre S x₀, F y

theorem marg_empty (F : bn.Assignment → R) (x₀ : bn.Assignment) : marg ∅ F x₀ = F x₀ := by
  simp [marg, fibre_empty]

theorem marg_univ (F : bn.Assignment → R) (x₀ : bn.Assignment) :
  marg Finset.univ F x₀ = ∑ y, F y := by
  simp [marg, fibre_univ]

theorem marg_insert {S : Finset bn.V} {v₀ : bn.V} (hv : v₀ ∉ S) (F : bn.Assignment → R)
  (x₀ : bn.Assignment) :
  marg (insert v₀ S) F x₀ = ∑ z, marg S F (Function.update x₀ v₀ z) := by
  unfold marg
  rw [← Finset.sum_fiberwise (fibre (insert v₀ S) x₀) (fun y ⇒ y v₀) F]
  refine Finset.sum_congr rfl fun z _ => ?_
  rw [fibre_insert_filter hv]

/-- Integrating out 'S ∪ T' (disjoint) is integrating out 'S' and then, inside, 'T': the finite
Fubini theorem for fibres. -/
theorem marg_union_disjoint {S T : Finset bn.V} (hd : Disjoint S T) (F : bn.Assignment → R)
  (x₀ : bn.Assignment) : marg (S ∪ T) F x₀ = marg S (fun y ⇒ marg T F y) x₀ := by
  induction S using Finset.induction_on generalizing x₀ with
  | empty => simp [marg_empty]
  | insert v₀ S hv ih =>
    have hvT : v₀ ∉ T := fun h => (Finset.disjoint_left.1 hd (Finset.mem_insert_self v₀ S)) h
    have hd' : Disjoint S T := hd.mono_left (Finset.subset_insert _ _)
    rw [Finset.insert_union, marg_insert (by simp [hv, hvT]), marg_insert hv]
    exact Finset.sum_congr rfl fun z _ => ih hd' _

/-- A factor that is constant on the fibre comes out of the sum. -/
theorem marg_mul_left {T : Finset bn.V} {F G : bn.Assignment → R} {x₀ : bn.Assignment}
  (hF : ∀ y ∈ fibre T x₀, F y = F x₀) :
  marg T (fun y ⇒ F y * G y) x₀ = F x₀ * marg T G x₀ := by
  unfold marg
  rw [Finset.mul_sum]
  exact Finset.sum_congr rfl fun y hy => by show F y * G y = F x₀ * G y; rw [hF y hy]

```

Partial joints along a list of variables

```

/-- The product of the kernels of the mechanisms whose target lies in 'l'. -/
def partialJoint (κ : bn.Kernel R) (l : List bn.V) (x : bn.Assignment) : R :=
  ∏ m ∈ Finset.univ.filter (fun m => bn.target m ∈ l), κ m x (x (bn.target m))

```

```

theorem filter_target_cons (hij : Function.Injective bn.target) {l : List bn.V} (m₀ : bn.M) :
  Finset.univ.filter (fun m ⇒ bn.target m ∈ bn.target m₀ :: l) =
    insert m₀ (Finset.univ.filter (fun m ⇒ bn.target m ∈ l)) := by
ext m
simp only [Finset.mem_filter, Finset.mem_univ, true_and, List.mem_cons, Finset.mem_insert]
constructor
· rintro (h | h)
· exact Or.inl (hij h)
· exact Or.inr h
· rintro (rfl | h)
· exact Or.inl rfl
· exact Or.inr h

/-- The core induction: integrating out the variables of a suffix 'l' of a topological order
(every variable in 'l' has a mechanism, every parent of a mechanism targeting 'l' that lies in
'l' comes earlier) gives total weight one, whatever the values of the other variables. -/
theorem marg_partialJoint_eq_one (κ : bn.Kernel R) (hij : Function.Injective bn.target)
(hloc : ∀ m, Local κ m)
(hself : ∀ m, bn.target m ∉ bn.parents m) (l : List bn.V) (hnd : l.Nodup)
(hnorm : ∀ m, bn.target m ∈ l → Normalised κ m)
(hpw : l.Pairwise (fun a b ⇒ ∀ m, bn.target m = a → b ∉ bn.parents m))
(hsurj : ∀ v ∈ l, ∃ m, bn.target m = v) (x₀ : bn.Assignment) :
  marg l.toFinset (partialJoint κ l) x₀ = 1 := by
induction l generalizing x₀ with
| nil ⇒ simp [marg_empty, partialJoint]
| cons v₀ l ih ⇒
  obtain ⟨m₀, hm₀⟩ := hsurj v₀ (List.mem_cons_self ..)
  subst hm₀
  rw [List.nodup_cons] at hnd
  rw [List.pairwise_cons] at hpw
  have hnotl : bn.target m₀ ∉ l.toFinset := by simp using hnd.1
  rw [List.toFinset_cons, marg_insert hnotl]
  have key : ∀ z, marg l.toFinset (partialJoint κ (bn.target m₀ :: l))
    (Function.update x₀ (bn.target m₀) z) = κ m₀ x₀ z := by
    intro z
    have hfac : ∀ y ∈ fibre l.toFinset (Function.update x₀ (bn.target m₀) z),
      partialJoint κ (bn.target m₀ :: l) y = κ m₀ x₀ z * partialJoint κ l y := by
      intro y hy
      rw [mem_fibre] at hy
      unfold partialJoint
      rw [filter_target_cons hij, Finset.prod_insert (by simp using hnd.1)]
      congr 1
      have hy0 : y (bn.target m₀) = z := by
        rw [hy _ hnotl, Function.update_self]
      have hk : κ m₀ y = κ m₀ x₀ := by
        refine hloc m₀ y x₀ fun p hp ⇒ ?_
        have hpl : p ∉ l := fun hpl ⇒ hpw.1 p hpl m₀ rfl hp
        have hne : p ≠ bn.target m₀ := fun h ⇒ hself m₀ (h ▸ hp)
        rw [hy p (by simp using hpl), Function.update_of_ne hne]
      rw [hk, hy0]
    unfold marg
    rw [Finset.sum_congr rfl hfac, ← Finset.mul_sum]
    have := ih hnd.2 (fun m hm ⇒ hnorm m (List.mem_cons_of_mem _ hm)) hpw.2
      (fun v hv ⇒ hsurj v (List.mem_cons_of_mem _ hv))
      (Function.update x₀ (bn.target m₀) z)
    unfold marg at this
    rw [this, mul_one]
  rw [Finset.sum_congr rfl fun z _ ⇒ key z]
  exact hnorm m₀ (List.mem_cons_self ..) x₀

```

Proposition 1

```

/-- **Proposition 1a.** In a closed network with a topological order, local and normalised
kernels make the joint a probability distribution. -/

```

```

theorem sum_joint_eq_one (κ : bn.Kernel R) (hclosed : bn.Closed) (ord : bn.TopoOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) :
  ∑ x, joint κ x = 1 := by
  obtain ⟨x₀⟩ : Nonempty bn.Assignment := inferInstance
  have h := marg_partialJoint_eq_one κ hclosed.1 hloc ord.no_self ord.order ord.nodup
    (fun m _ => hnorm m) ord.parents_before (fun v _ => hclosed.2 v) x₀
  have hu : ord.order.toFinset = Finset.univ :=
    Finset.eq_univ_of_forall fun v => by simp using ord.complete v
  rw [hu, marg_univ] at h
  rw [← h]
  refine Finset.sum_congr rfl fun x _ => ?_
  unfold joint partialJoint
  rw [Finset.filter_true_of_mem fun m _ => ord.complete _]

/-- The sequential evaluator: walk the variables in the given order and multiply in the kernels
of the mechanisms generating each of them. This is the loop 'for v in order' of
'joint_distribution' in Julia. -/
def evalSeq (κ : bn.Kernel R) (l : List bn.V) (x : bn.Assignment) : R :=
  (l.map fun v => ∏ m ∈ Finset.univ.filter (fun m => bn.target m = v), κ m x (x (bn.target m))).prod

/-- **Proposition 1b.** The sequential evaluator along any complete duplicate-free order of the
variables computes the joint product (no acyclicity or closedness is needed for this identity). -/
theorem evalSeq_eq_joint (κ : bn.Kernel R) (l : List bn.V) (hnd : l.Nodup) (hcomp : ∀ v, v ∈ l)
  (x : bn.Assignment) : evalSeq κ l x = joint κ x := by
  unfold evalSeq joint
  have hu : l.toFinset = Finset.univ := Finset.eq_univ_of_forall fun v => by simp using hcomp v
  rw [← List.prod_toFinset _ hnd, hu]
  exact Finset.prod_fiberwise Finset.univ bn.target _

/-- Proposition 1b specialised to a topological order. -/
theorem evalSeq_topo_eq_joint (κ : bn.Kernel R) (ord : bn.TopoOrder) (x : bn.Assignment) :
  evalSeq κ ord.order x = joint κ x :=
  evalSeq_eq_joint κ ord.order ord.nodup ord.complete x

```

Marginalising the downstream variables out (Proposition 3, closed-world shadow)

```

/-- A set of "upstream" variables 'U' is *closed under parents*: every mechanism generating a
variable of 'U' reads only variables of 'U'. -/
def UpstreamClosed (U : Finset bn.V) : Prop :=
  ∀ m, bn.target m ∈ U → bn.parents m ⊆ U

/-- **Integrating the downstream variables out.** If 'U' is closed under parents, every variable
outside 'U' is generated by some mechanism, and the kernels of those mechanisms are normalised,
then summing the joint over all assignments that agree with 'x₀' on 'U' leaves exactly the
product of the kernels of the mechanisms targeting 'U'. Only the *downstream* kernels have to be
normalised; the upstream ones merely have to be local.

This is the closed-world shadow of **Proposition 3** (SPEC §61): the downstream half of a
sequential composite integrates out to one, so the marginal of '[B ∘ A]' on 'A''s variables is
'[A]'. See 'BayesianNetworksProofs.Finite.Open' for the open-network form
('OpenFinBayesNet.Composable.marg_joint_compose'). -/
theorem marg_joint_downstream (κ : bn.Kernel R) (hinj : Function.Injective bn.target)
  (ord : bn.TopoOrder) (hloc : ∀ m, Local κ m) (U : Finset bn.V)
  (hnorm : ∀ m, bn.target m ∉ U → Normalised κ m)
  (hU : UpstreamClosed U) (hmech : ∀ v ∉ U, ∃ m, bn.target m = v) (x₀ : bn.Assignment) :
  marg Uᶜ (joint κ) x₀ =
  ∏ m ∈ Finset.univ.filter (fun m => bn.target m ∈ U), κ m x₀ (x₀ (bn.target m)) := by
  set l := ord.order.filter (fun v => decide (v ∉ U)) with hL
  have hmemL : ∀ v, v ∈ l ↔ v ∉ U := by
    intro v
    rw [hL, List.mem_filter]
    simp [ord.complete v]
  have hlt : l.toFinset = Uᶜ := by
    ext v

```

```

    simp [hmemL v]
  have key := marg_partialJoint_eq_one κ hinj hloc ord.no_self l (ord.nodup.filter _)
    (fun m hm => hnrm m ((hmemL _).1 hm)) (ord.parents_before.filter _)
    (fun v hv => hmech v ((hmemL v).1 hv)) x₀
  rw [← hlt]
  unfold marg at key ⊢
  have hfac : ∀ y ∈ fibre l.toFinset x₀, joint κ y =
    (∏ m ∈ Finset.univ.filter (fun m => bn.target m ∈ U), κ m x₀ (x₀ (bn.target m)))
    * partialJoint κ l y := by
    intro y hy
    rw [mem_fibre] at hy
    have hagree : ∀ v ∈ U, y v = x₀ v := fun v hv =>
      hy v (by simp [List.mem_toFinset, hmemL v, hv])
    unfold joint partialJoint
    rw [← Finset.prod_filter_mul_prod_filter_not Finset.univ (fun m => bn.target m ∈ U)]
    congr 1
    · refine Finset.prod_congr rfl fun m hm => ?_
      have hmU : bn.target m ∈ U := (Finset.mem_filter.1 hm).2
      rw [hloc m y x₀ fun p hp => hagree p (hU m hmU hp), hagree _ hmU]
    · exact Finset.prod_congr (by ext m; simp [hmemL]) fun m _ => rfl
  rw [Finset.sum_congr rfl hfac, ← Finset.mul_sum, key, mul_one]

/-- **Proposition 3 (closed-world shadow).** In a closed network with a topological order and
local, normalised kernels, integrating the downstream variables out of the full joint leaves the
joint of the upstream mechanisms. -/
theorem marg_joint_upstream (κ : bn.Kernel R) (hclosed : bn.Closed) (ord : bn.TopoOrder)
  (hloc : ∀ m, Local κ m) (hnrm : ∀ m, Normalised κ m) (U : Finset bn.V)
  (hU : UpstreamClosed U) (x₀ : bn.Assignment) :
  marg Uᶜ (joint κ) x₀ =
  ∏ m ∈ Finset.univ.filter (fun m => bn.target m ∈ U), κ m x₀ (x₀ (bn.target m)) :=
  marg_joint_downstream κ hclosed.1 ord hloc U (fun m _ => hnrm m) hU
  (fun v _ => hclosed.2 v) x₀

end FinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.Intervention

```
import BayesianNetworksProofs.Finite.Evaluation
```

Proposition 4 (hard intervention) for the concrete finite model (SPEC §21.2, §22).

do($X = a$) replaces the mechanism m_θ generating X by the point mass δ_a , ignoring its former parents. Syntactically this is cut bn m_θ (the Input rows of m_θ are deleted, i.e. $\text{parents } m_\theta := \emptyset$); semantically it is intervene $\kappa m_\theta a$, which overwrites the kernel of m_θ with the indicator of a .

- joint_intervene — the truncated factorisation: the intervened joint is the indicator $[x \ X = a]$ times the product of the *other* mechanisms' kernels, which are untouched.
- normalised_intervene, local_cut — the intervened family is again normalised and local for the rewritten network, so Proposition 1 applies to it (sum_joint_intervene_eq_one).

```

namespace BayesianNetworksProofs

namespace FinBayesNet

variable {bn : FinBayesNet} {R : Type}

/-- The syntactic rewrite 'do': the mechanism 'm_θ' keeps its target but loses its inputs. -/
def cut (bn : FinBayesNet) (m_θ : bn.M) : FinBayesNet :=
  { bn with parents := Function.update bn.parents m_θ ∅ }

@[simp] theorem cut_parents (m_θ m : bn.M) :
  (bn.cut m_θ).parents m = Function.update bn.parents m_θ ∅ m := rfl

```

```

/-- A topological order of 'bn' is one of 'bn.cut m₀' (parents only shrink). -/
def TopoOrder.cut (ord : bn.TopoOrder) (m₀ : bn.M) : (bn.cut m₀).TopoOrder where
  order := ord.order
  nodup := ord.nodup
  complete := ord.complete
  parents_before := ord.parents_before.imp fun {a b} h (m : bn.M) hm => by
    show b ∉ Function.update bn.parents m₀ ∅ m
    by_cases hmm : m = m₀
    · subst hmm
      simp
    · rw [Function.update_of_ne hmm]
      exact h m hm
  no_self := fun (m : bn.M) => by
    show bn.target m ∉ Function.update bn.parents m₀ ∅ m
    by_cases hmm : m = m₀
    · subst hmm
      simp
    · rw [Function.update_of_ne hmm]
      exact ord.no_self m

theorem closed_cut (hclosed : bn.Closed) (m₀ : bn.M) : (bn.cut m₀).Closed := hclosed

variable [CommSemiring R]

/-- The semantic rewrite: replace the kernel of 'm₀' by the point mass at 'a'. -/
def intervene (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀)) : bn.Kernel R :=
  Function.update κ m₀ (fun _ y => if y = a then 1 else 0)

@[simp] theorem intervene_self (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀))
  (x : bn.Assignment) (y : bn.states (bn.target m₀)) :
  intervene κ m₀ a m₀ x y = if y = a then 1 else 0 := by
  simp [intervene]

theorem intervene_of_ne (κ : bn.Kernel R) {m₀ m : bn.M} (h : m ≠ m₀)
  (a : bn.states (bn.target m₀)) : intervene κ m₀ a m = κ m := by
  simp [intervene, Function.update_of_ne h]

/-- **Proposition 4** (truncated factorisation). -/
theorem joint_intervene (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀))
  (x : bn.Assignment) :
  joint (intervene κ m₀ a) x =
    (if x (bn.target m₀) = a then 1 else 0) *
    ∏ m ∈ Finset.univ.erase m₀, κ m x (x (bn.target m)) := by
  unfold joint
  rw [← Finset.mul_prod_erase Finset.univ _ (Finset.mem_univ m₀)]
  congr 1
  · simp
  · exact Finset.prod_congr rfl fun m hm => by
    rw [intervene_of_ne κ (Finset.ne_of_mem_erase hm)]

/-- The point-mass kernel is normalised. -/
theorem normalised_intervene_self (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀)) :
  Normalised (intervene κ m₀ a) m₀ := by
  intro x
  simp

/-- Intervening preserves normalisation of every mechanism. -/
theorem normalised_intervene (κ : bn.Kernel R) (hnorm : ∀ m, Normalised κ m) (m₀ : bn.M)
  (a : bn.states (bn.target m₀)) : ∀ m, Normalised (intervene κ m₀ a) m := by
  intro m
  by_cases h : m = m₀
  · subst h
    exact normalised_intervene_self κ m a
  · intro x
    rw [intervene_of_ne κ h]

```

```

    exact hnorm m x

/-- The point-mass kernel reads no parents at all. -/
theorem localOn_empty_intervene (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀)) :
  LocalOn ∅ (intervene κ m₀ a m₀) := by
  intro x x' _
  funext y
  simp

/-- The intervened family is local for the rewritten network 'bn.cut m₀'. -/
theorem local_cut (κ : bn.Kernel R) (hloc : ∀ m, Local κ m) (m₀ : bn.M)
  (a : bn.states (bn.target m₀)) :
  ∀ m, Local (bn := bn.cut m₀) (intervene κ m₀ a) m := by
  refine fun (m : bn.M) (x x' : bn.Assignment) hx => ?_
  by_cases h : m = m₀
  · subst h
    exact localOn_empty_intervene κ m a x x' (fun p hp => absurd hp (Finset.notMem_empty p))
  · show intervene κ m₀ a m x = intervene κ m₀ a m x'
    rw [intervene_of_ne κ h]
    refine hloc m x x' fun p hp => hx p ?_
    show p ∈ Function.update bn.parents m₀ ∅ m
    rw [Function.update_of_ne h]
    exact hp

/-- The intervened joint is again a probability distribution (Proposition 1 applied to the
rewritten network). -/
theorem sum_joint_intervene_eq_one (κ : bn.Kernel R) (hclosed : bn.Closed) (ord : bn.TopoOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (m₀ : bn.M)
  (a : bn.states (bn.target m₀)) :
  ∑ x, joint (intervene κ m₀ a) x = 1 :=
  sum_joint_eq_one (bn := bn.cut m₀) (intervene κ m₀ a) (closed_cut hclosed m₀) (ord.cut m₀)
  (local_cut κ hloc m₀ a) (normalised_intervene κ hnorm m₀ a)

end FinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.Tensor

```

import BayesianNetworksProofs.Finite.Evaluation
import Mathlib.Data.Fintype.Sum
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Data.Finset.Sum

```

Proposition 2 (tensor compositionality) for the concrete finite model (SPEC §13.1).

The tensor $bn_1.tensor\ bn_2$ of two networks has variables $V_1 \oplus V_2$ and mechanisms $M_1 \oplus M_2$ (the disjoint union of the two ACSets, $\oplus$ of OpenBayesNets with no shared feet). Its kernels are the two families side by side, and `joint_tensor` shows that the joint of the tensor is the product of the two joints — $\llbracket A \otimes B \rrbracket = \llbracket A \rrbracket \otimes \llbracket B \rrbracket$ in the finite stochastic semantics. The tensor of closed / local / normalised / topologically ordered networks is again so.

```

namespace BayesianNetworksProofs

namespace FinBayesNet

/-- Disjoint union of two networks. Reducible so that 'simp'/'rw' see 'V = V₁ ⊕ V₂' etc. -/
@[reducible] def tensor (bn₁ bn₂ : FinBayesNet) : FinBayesNet where
  V := bn₁.V ⊕ bn₂.V
  M := bn₁.M ⊕ bn₂.M
  states := Sum.elim bn₁.states bn₂.states
  fintypeS := fun
    | .inl v => bn₁.fintypeS v
    | .inr v => bn₂.fintypeS v
  decS := fun

```

```

    | .inl v ⇒ bn1.decS v
    | .inr v ⇒ bn2.decS v
  nonemptyS := fun
    | .inl v ⇒ bn1.nonemptyS v
    | .inr v ⇒ bn2.nonemptyS v
  target := Sum.map bn1.target bn2.target
  parents := fun
    | .inl m ⇒ (bn1.parents m).map Function.Embedding.inl
    | .inr m ⇒ (bn2.parents m).map Function.Embedding.inr

variable {bn1 bn2 : FinBayesNet} {R : Type}

@[simp] theorem tensor_target_inl (m : bn1.M) :
  (bn1.tensor bn2).target (.inl m) = .inl (bn1.target m) := rfl

@[simp] theorem tensor_target_inr (m : bn2.M) :
  (bn1.tensor bn2).target (.inr m) = .inr (bn2.target m) := rfl

@[simp] theorem tensor_parents_inl (m : bn1.M) :
  (bn1.tensor bn2).parents (.inl m) = (bn1.parents m).map Function.Embedding.inl := rfl

@[simp] theorem tensor_parents_inr (m : bn2.M) :
  (bn1.tensor bn2).parents (.inr m) = (bn2.parents m).map Function.Embedding.inr := rfl

@[simp] theorem inl_mem_map_inl {α β : Type} {s : Finset α} {a : α} :
  Sum.inl a ∈ s.map (Function.Embedding.inl (β := β)) ↔ a ∈ s := by
  simp [Finset.mem_map]

@[simp] theorem inr_mem_map_inr {α β : Type} {s : Finset β} {b : β} :
  Sum.inr b ∈ s.map (Function.Embedding.inr (α := α)) ↔ b ∈ s := by
  simp [Finset.mem_map]

@[simp] theorem inl_notMem_map_inl {α β : Type} {s : Finset α} {b : β} :
  Sum.inr b ∉ s.map (Function.Embedding.inl (β := β)) := by
  simp [Finset.mem_map]

@[simp] theorem inl_notMem_map_inr {α β : Type} {s : Finset β} {a : α} :
  Sum.inl a ∉ s.map (Function.Embedding.inr (α := α)) := by
  simp [Finset.mem_map]

/-- Restrict an assignment of the tensor to the left factor. -/
def Assignment.left (x : (bn1.tensor bn2).Assignment) : bn1.Assignment := fun v ⇒ x (.inl v)

/-- Restrict an assignment of the tensor to the right factor. -/
def Assignment.right (x : (bn1.tensor bn2).Assignment) : bn2.Assignment := fun v ⇒ x (.inr v)

/-- Two kernel families side by side. -/
def tensorKernel (κ1 : bn1.Kernel R) (κ2 : bn2.Kernel R) : (bn1.tensor bn2).Kernel R := fun
  | .inl m ⇒ fun x y ⇒ κ1 m (Assignment.left x) y
  | .inr m ⇒ fun x y ⇒ κ2 m (Assignment.right x) y

/-- Proposition 2. The joint of the tensor is the product of the joints. -/
theorem joint_tensor [CommMonoid R] (κ1 : bn1.Kernel R) (κ2 : bn2.Kernel R)
  (x : (bn1.tensor bn2).Assignment) :
  joint (tensorKernel κ1 κ2) x = joint κ1 (Assignment.left x) * joint κ2 (Assignment.right x) := by
  unfold joint
  exact Fintype.prod_sum_type _

theorem closed_tensor (h1 : bn1.Closed) (h2 : bn2.Closed) : (bn1.tensor bn2).Closed :=
  h1.sumMap h2

theorem normalised_tensor [AddCommMonoid R] [One R] (κ1 : bn1.Kernel R) (κ2 : bn2.Kernel R)
  (h1 : ∀ m, Normalised κ1 m) (h2 : ∀ m, Normalised κ2 m) :
  ∀ m, Normalised (tensorKernel κ1 κ2) m
  | .inl m ⇒ fun x ⇒ h1 m (Assignment.left x)
  | .inr m ⇒ fun x ⇒ h2 m (Assignment.right x)

```

```

theorem local_tensor (κ1 : bn1.Kernel R) (κ2 : bn2.Kernel R)
  (h1 : ∀ m, Local κ1 m) (h2 : ∀ m, Local κ2 m) :
  ∀ m, Local (tensorKernel κ1 κ2) m
| .inl m ⇒ fun x x' hx ⇒
  h1 m (Assignment.left x) (Assignment.left x') fun p hp ⇒
  hx (.inl p) (Finset.mem_map_of_mem _ hp)
| .inr m ⇒ fun x x' hx ⇒
  h2 m (Assignment.right x) (Assignment.right x') fun p hp ⇒
  hx (.inr p) (Finset.mem_map_of_mem _ hp)

/-- Concatenating topological orders gives a topological order of the tensor. -/
def TopoOrder.tensor (o1 : bn1.TopoOrder) (o2 : bn2.TopoOrder) : (bn1.tensor bn2).TopoOrder where
order := o1.order.map Sum.inl ++ o2.order.map Sum.inr
nodup := by
  refine List.Nodup.append (o1.nodup.map Sum.inl_injective) (o2.nodup.map Sum.inr_injective) ?_
  intro a h1 h2
  obtain ⟨v, _, rfl⟩ := List.mem_map.1 h1
  obtain ⟨w, _, h⟩ := List.mem_map.1 h2
  exact Sum.inr_ne_inl h
complete := fun
| .inl v ⇒ List.mem_append_left _ (List.mem_map_of_mem (o1.complete v))
| .inr v ⇒ List.mem_append_right _ (List.mem_map_of_mem (o2.complete v))
parents_before := by
  refine List.pairwise_append.2 ⟨List.pairwise_map.2 (o1.parents_before.imp ?_),
    List.pairwise_map.2 (o2.parents_before.imp ?_), ?_⟩
  • rintro a b h (m | m) hm
    • simp using h m (Sum.inl_injective hm)
    • simp at hm
  • rintro a b h (m | m) hm
    • simp at hm
    • simp using h m (Sum.inr_injective hm)
  • rintro a ha b hb (m | m) hm
    • obtain ⟨w, _, rfl⟩ := List.mem_map.1 hb
      simp
    • obtain ⟨v, _, rfl⟩ := List.mem_map.1 ha
      simp at hm
no_self := fun
| .inl m ⇒ by simp using o1.no_self m
| .inr m ⇒ by simp using o2.no_self m

end FinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.Open

```
import BayesianNetworksProofs.Finite.Tensor
```

Open networks and the closure theorem (SPEC §13 and its revision note, §55.5, §61 Propositions 2 and 3), for the concrete finite model — no category theory, no quotient types.

An `OpenFinBayesNet` is a `FinBayesNet` with two distinguished sets of variables, inputs and outputs, satisfying the *typed-interface rule* of `BayesianNetworks.jl`'s `Open(src/open.jl, validation_errors (:: OpenBayesNet))`:

Julia rule	Lean field
at most one mechanism per variable (<code>validate(bn; closed = false)</code>)	<code>target_inj</code>
rule 1: input-foot variables have no mechanism	<code>input_exogenous</code>
rule 3: every mechanism-free apex variable is an input	<code>exogenous_input</code>
rule 4: the derived graph is acyclic	<code>topo</code> (a <code>TopoOrder</code>)

Rule 2 (the input leg is injective) and rule 5 (legs natural, i.e. name-, reference- and state-preserving) are absorbed into the representation: the feet here are *subsets* of the apex variables rather than separate objects with a leg, so the leg is the inclusion, which is injective and natural by construction. The interface match of `compose` is the injection ι below, which plays the part of Catlab’s `interface_matches` and of the pushout’s gluing map; `states_equiv` is the “same states, same positions” half of `interface_matches`.

Composition without quotients

`Composable A B` is an injection ι from B’s inputs into A’s outputs together with the matching of their state spaces. The pushout of the two structured cospans identifies each input of B with its partner output of A; rather than quotient $A.V \oplus B.V$, the composite takes

- $V := A.V \oplus \text{Priv } B$ where $\text{Priv } B = \{v : B.V \mid v \notin B.\text{inputs}\}$ — one representative per class, the glued variables being represented on the A side;
- $M := A.M \oplus B.M$ — mechanisms are never identified;
- $\text{tr} : B.V \rightarrow A.V \oplus \text{Priv } B$, injective, sending an input of B to `Sum.inl` ($\iota \dots$) and any other variable to `Sum.inr` — the right-hand pushout leg.

`composeNet` is the resulting shape and `compose` bundles it back into an `OpenFinBayesNet`, which is the closure theorem: producing the four fields *is* the proof that the composite is again valid. The individual statements are

- `composeNet_target_injective` — **at most one mechanism per variable is preserved**. The key step is that a mechanism of B never targets a glued variable, because B’s inputs are exogenous (`target_notMem_inputs`); so the two mechanism families cannot collide;
- `compose_input_exogenous`, `compose_exogenous_input` — the composite’s inputs are exactly `A.inputs`, and every variable contributed by B keeps its mechanism;
- `composeTopo` — **acyclicity is preserved**: A’s order followed by B’s order restricted to `Priv B` is a topological order of the composite. It is *derived*, not assumed.

Outputs are $(A.\text{outputs} \setminus \text{glued}) \cup \text{tr} (B.\text{outputs})$, exactly as in Julia’s glue.

Semantics (Proposition 3)

`composeKernel` puts two kernel families side by side; `restrictA` and `restrictB` read an assignment of the composite as an assignment of A and of B (a glued variable is read on the A side and transported by `states_equiv`).

- `joint_compose` — $\llbracket B \circ A \rrbracket = \llbracket A \rrbracket \cdot \llbracket B \rrbracket$ pointwise: the joint of the composite factors as the product of the two joints. This is the sequential analogue of `Finite/Tensor.lean`’s `joint_tensor`, and the algebraic heart of Proposition 3;
- `local_compose`, `normalised_compose`, `closed_compose`, `sum_joint_compose_eq_one` — the composite of local / normalised kernels is local / normalised, and if A has no inputs the composite is closed, so its joint is a probability distribution;
- `marg_joint_compose` — summing the composite joint over all values of B’s private variables returns A’s joint: $\llbracket B \circ A \rrbracket$ marginalised back onto A is $\llbracket A \rrbracket$. Only B’s kernels need to be normalised;
- `marg_joint_compose_split` — **Proposition 3**. Integrating out $S \cup T$, with S on the A side and T on the B side, is $\sum_S \llbracket A \rrbracket \cdot (\sum_T \llbracket B \rrbracket)$: the composite semantics is the sum over the interface of the product of the two open semantics. Taking S to be A’s hidden variables together with the glued interface and T to be B’s hidden variables gives $\llbracket B \circ A \rrbracket = \llbracket B \rrbracket \circ \llbracket A \rrbracket$. What is *not* proved here is the last book-keeping step that rewrites those two marginals as the open semantics of A and of B on their own variable types; see `Roadmap.lean`.

```
namespace BayesianNetworksProofs
```

```
structure OpenFinBayesNet extends FinBayesNet where
  inputs : Finset V
  outputs : Finset V
  target_inj : Function.Injective target
  input_exogenous : ∀ v ∈ inputs, ∀ m, target m ≠ v
  exogenous_input : ∀ v, (∀ m, target m ≠ v) → v ∈ inputs
  topo : FinBayesNet.TopoOrder toFinBayesNet
```

```
namespace OpenFinBayesNet
```

```
open FinBayesNet
```

```

variable {A B : OpenFinBayesNet}

/-- Rule 1 restated: a mechanism never targets an input. -/
theorem target_notMem_inputs (O : OpenFinBayesNet) (m : O.M) : O.target m  $\notin$  O.inputs :=
  fun h  $\Rightarrow$  O.input_exogenous _ h m rfl

/-- Rule 3 restated: a non-input variable is generated by some mechanism. -/
theorem exists_mechanism (O : OpenFinBayesNet) {v : O.V} (h : v  $\notin$  O.inputs) :
   $\exists$  m, O.target m = v := by
  by_contra hc
  exact h (O.exogenous_input v (fun m hm  $\Rightarrow$  hc <m, hm>))

/-- The hidden (non-interface) variables of an open network. -/
def hidden (O : OpenFinBayesNet) : Finset O.V := (O.inputs  $\cup$  O.outputs)c

/-- The open semantics '[O]' of an open network: the joint with the hidden variables summed
out, a function of the interface (input and output) variables alone. -/
def osem {R : Type} [CommSemiring R] (O : OpenFinBayesNet) ( $\kappa$  : O.Kernel R)
  (x : O.Assignment) : R := marg O.hidden (joint  $\kappa$ ) x

/-- The variables of 'B' that are not inputs – one representative per pushout class on the
'B' side. -/
abbrev Priv (B : OpenFinBayesNet) := {v : B.V // v  $\notin$  B.inputs}

/-- 'some' on the non-input variables of 'B', 'none' on the inputs. -/
def privOf (v : B.V) : Option (Priv B) := if h : v  $\in$  B.inputs then none else some <v, h>

/-- 'privOf' picks out exactly the non-input variables. -/
theorem privOf_eq_some {v : B.V} {w : Priv B} : privOf v = some w  $\Leftrightarrow$  v = w.val := by
  unfold privOf
  by_cases h : v  $\in$  B.inputs
  · simp only [dif_pos h, reduceCtorEq, false_iff]
    rintro rfl
    exact w.property h
  · simp [dif_neg h, Subtype.ext_iff]

/-- An interface match: an injection of 'B''s inputs into 'A''s outputs (Catlab's gluing map,
validated by 'interface_matches' in Julia) together with the identification of the state spaces
it pairs up. -/
structure Composable (A B : OpenFinBayesNet) where
   $\iota$  : {v : B.V // v  $\in$  B.inputs}  $\rightarrow$  {v : A.V // v  $\in$  A.outputs}
   $\iota$ _inj : Function.Injective  $\iota$ 
  states_equiv :  $\forall$  w : {v : B.V // v  $\in$  B.inputs}, A.states ( $\iota$  w).val  $\simeq$  B.states w.val

namespace Composable

/-- The right-hand pushout leg: 'B''s variables inside the composite. -/
def tr (c : Composable A B) (v : B.V) : A.V  $\oplus$  Priv B :=
  if h : v  $\in$  B.inputs then Sum.inl (c. $\iota$  <v, h>).val else Sum.inr <v, h>

variable {c : Composable A B}

theorem tr_of_mem {v : B.V} (h : v  $\in$  B.inputs) : c.tr v = Sum.inl (c. $\iota$  <v, h>).val := dif_pos h

theorem tr_of_notMem {v : B.V} (h : v  $\notin$  B.inputs) : c.tr v = Sum.inr <v, h> := dif_neg h

/-- The right leg is injective: two 'B' variables are identified in the composite only if they
were equal. -/
theorem tr_injective : Function.Injective c.tr := by
  intro v w h
  by_cases hv : v  $\in$  B.inputs <=> by_cases hw : w  $\in$  B.inputs
  · rw [tr_of_mem hv, tr_of_mem hw, Sum.inl.injEq] at h
    exact congrArg Subtype.val (c. $\iota$ _inj (Subtype.ext h))
  · rw [tr_of_mem hv, tr_of_notMem hw] at h
    exact absurd h (by simp)

```

```

• rw [tr_of_notMem hv, tr_of_mem hw] at h
  exact absurd h (by simp)
• rw [tr_of_notMem hv, tr_of_mem hw, Sum.inr.injEq] at h
  exact congrArg Subtype.val h

variable (c) in
/-- 'tr' as an embedding, for 'Finset.map'. -/
def trEmb : B.V  $\hookrightarrow$  A.V  $\otimes$  Priv B := (c.tr, tr_injective)

@[simp] theorem trEmb_apply (v : B.V) : c.trEmb v = c.tr v := rfl

@[simp] theorem inr_mem_map_trEmb {s : Finset B.V} {w : Priv B} :
  Sum.inr w  $\in$  s.map c.trEmb  $\leftrightarrow$  w.val  $\in$  s := by
  simp only [Finset.mem_map, trEmb_apply]
  constructor
  • rintro (p, hp, h)
    by_cases hpi : p  $\in$  B.inputs
    • rw [tr_of_mem hpi] at h
      exact absurd h (by simp)
    • rw [tr_of_notMem hpi, Sum.inr.injEq] at h
      rw [← h]
      exact hp
  • exact fun h  $\Rightarrow$  (w.val, h, tr_of_notMem w.property)

variable (c) in
/-- The 'A'-outputs that are glued to inputs of 'B'. -/
def glued : Finset A.V := Finset.univ.image (fun w : {v : B.V // v  $\in$  B.inputs}  $\Rightarrow$  (c.l w).val)

variable (c) in
/-- The underlying shape of the composite. -/
@[reducible] def composeNet : FinBayesNet where
  V := A.V  $\otimes$  Priv B
  M := A.M  $\otimes$  B.M
  states := Sum.elim A.states (fun w  $\Rightarrow$  B.states w.val)
  fintypeS := fun
    | .inl v  $\Rightarrow$  A.fintypeS v
    | .inr w  $\Rightarrow$  B.fintypeS w.val
  decS := fun
    | .inl v  $\Rightarrow$  A.decS v
    | .inr w  $\Rightarrow$  B.decS w.val
  nonemptyS := fun
    | .inl v  $\Rightarrow$  A.nonemptyS v
    | .inr w  $\Rightarrow$  B.nonemptyS w.val
  target := Sum.elim (fun m  $\Rightarrow$  Sum.inl (A.target m))
    (fun m  $\Rightarrow$  Sum.inr (B.target m, B.target_notMem_inputs m))
  parents := Sum.elim (fun m  $\Rightarrow$  (A.parents m).map Function.Embedding.inl)
    (fun m  $\Rightarrow$  (B.parents m).map c.trEmb)

-- 'composeNet' is reducible, so instance search sees through '(composeNet c).states v' to the
-- 'Sum.elim' and no longer matches 'FinBayesNet.fintypeS'; re-expose the instances by hand.
instance instFintypeComposeStates (v : (composeNet c).V) : Fintype ((composeNet c).states v) :=
  (composeNet c).fintypeS v

instance instDecidableEqComposeStates (v : (composeNet c).V) :
  DecidableEq ((composeNet c).states v) := (composeNet c).decS v

@[simp] theorem composeNet_target_inl (m : A.M) :
  (composeNet c).target (.inl m) = .inl (A.target m) := rfl

@[simp] theorem composeNet_target_inr (m : B.M) :
  (composeNet c).target (.inr m) = .inr (B.target m, B.target_notMem_inputs m) := rfl

@[simp] theorem composeNet_parents_inl (m : A.M) :
  (composeNet c).parents (.inl m) = (A.parents m).map Function.Embedding.inl := rfl

@[simp] theorem composeNet_parents_inr (m : B.M) :

```

```

    (composeNet c).parents (.inr m) = (B.parents m).map c.trEmb := rfl

variable (c) in
/-- Appending 'B''s topological order (on its non-input variables) after 'A''s. -/
def composeTopo : (composeNet c).TopoOrder where
  order := A.topo.order.map Sum.inl ++ (B.topo.order.filterMap privOf).map Sum.inr
  nodup := by
    refine List.Nodup.append (A.topo.nodup.map Sum.inl_injective)
      (List.Nodup.map Sum.inr_injective ?_) ?_
    • refine List.Nodup.filterMap ?_ B.topo.nodup
      intro a a' b hb hb'
      rw [Option.mem_def, privOf_eq_some] at hb hb'
      exact hb.trans hb'.symm
    • rintro x hx hx'
      obtain ⟨a, _, rfl⟩ := List.mem_map.1 hx
      obtain ⟨w, _, h⟩ := List.mem_map.1 hx'
      exact Sum.inr_ne_inl h
  complete := fun
    | .inl a ⇒ List.mem_append_left _ (List.mem_map_of_mem (A.topo.complete a))
    | .inr w ⇒ List.mem_append_right _
      (List.mem_map_of_mem (List.mem_filterMap.2 ⟨w.val, B.topo.complete w.val,
        privOf_eq_some.2 rfl⟩))
  parents_before := by
    refine List.pairwise_append.2 (?.?.?.)
    • refine List.pairwise_map.2 (A.topo.parents_before.imp ?_)
      rintro a b hab (m | m) hm
      • rw [composeNet_target_inl, Sum.inl.injEq] at hm
        simp using hab m hm
      • exact absurd hm (by simp)
    • have hb : ((B.topo.order.filterMap privOf).Pairwise
      fun w w' : Priv B ⇒ ∀ m, B.target m = w.val → w'.val ∉ B.parents m) := by
      refine List.Pairwise.filterMap privOf ?_ B.topo.parents_before
      intro a a' hR b hb b' hb'
      rw [privOf_eq_some] at hb hb'
      subst hb; subst hb'
      exact hR
      refine List.pairwise_map.2 (hb.imp ?_)
      • rintro w w' hww' (m | m) hm
        • exact absurd hm (by simp)
        • rw [composeNet_target_inr, Sum.inr.injEq] at hm
          rw [composeNet_parents_inr, inr_mem_map_trEmb]
          exact hww' m (congrArg Subtype.val hm)
      • rintro x hx y hy (m | m) hm
        • obtain ⟨w, _, rfl⟩ := List.mem_map.1 hy
          simp
        • obtain ⟨a, _, rfl⟩ := List.mem_map.1 hx
          exact absurd hm (by simp)
  no_self := fun
    | .inl m ⇒ by simp using A.topo.no_self m
    | .inr m ⇒ by
      rw [composeNet_target_inr, composeNet_parents_inr, inr_mem_map_trEmb]
      exact B.topo.no_self m

variable (c) in
/-- **The closure theorem, rule 1 ("at most one mechanism per variable").** The target map of
the composite is injective whenever those of 'A' and 'B' are. -/
theorem composeNet_target_injective : Function.Injective (composeNet c).target := by
  rintro (m | m) (m' | m') h
  • exact congrArg Sum.inl (A.target_inj (Sum.inl.inj h))
  • exact absurd h (by simp)
  • exact absurd h (by simp)
  • exact congrArg Sum.inr (B.target_inj (congrArg Subtype.val (Sum.inr.inj h)))

variable (c) in
/-- **The closure theorem, rule 1' (inputs are exogenous).** No composite mechanism targets a
variable of 'A.inputs'. -/

```

```

theorem compose_input_exogenous :
  ∀ v ∈ A.inputs.map Function.Embedding.inl, ∀ m, (composeNet c).target m ≠ v := by
  intro v hv m
  obtain ⟨a, ha, rfl⟩ := Finset.mem_map.1 hv
  cases m with
  | inl m ⇒ exact fun h ⇒ A.input_exogenous a ha m (Sum.inl.inj h)
  | inr m ⇒ exact fun h ⇒ by simp at h

variable (c) in
/-- **The closure theorem, rule 3 (every mechanism-free variable is an input).** A composite
variable with no generating mechanism comes from 'A' and is an input of 'A'; the variables
contributed by 'B' all keep their mechanism. -/
theorem compose_exogenous_input :
  ∀ v, (∀ m, (composeNet c).target m ≠ v) → v ∈ A.inputs.map Function.Embedding.inl := by
  rintro (a | w) h
  · exact Finset.mem_map_of_mem _
    (A.exogenous_input a fun m hm ⇒ h (.inl m) (congrArg Sum.inl hm))
  · exact absurd (B.exogenous_input w.val fun m hm ⇒
    h (.inr m) (congrArg Sum.inr (Subtype.ext hm))) w.property

variable (c) in
/-- **The open-network closure theorem (SPEC §13, revision note).** The pushout composite of two
valid open networks along an injective interface match is again a valid open network: its inputs
are 'A''s inputs, its outputs are 'A''s unglued outputs together with 'B''s outputs, and the four
typed-interface rules hold. -/
@[reducible] def compose : OpenFinBayesNet where
  toFinBayesNet := composeNet c
  inputs := A.inputs.map Function.Embedding.inl
  outputs := (A.outputs \ c.glued).map Function.Embedding.inl ∪ B.outputs.map c.trEmb
  target_inj := composeNet_target_injective c
  input_exogenous := compose_input_exogenous c
  exogenous_input := compose_exogenous_input c
  topo := composeTopo c

```

Semantics

```

variable (c) in
/-- The 'A'-part of a composite assignment. -/
def restrictA (x : (composeNet c).Assignment) : A.Assignment := fun a ⇒ x (Sum.inl a)

variable (c) in
/-- The 'B'-part of a composite assignment: glued variables are read off the 'A' side
through the interface equivalence. -/
def restrictB (x : (composeNet c).Assignment) : B.Assignment := fun v ⇒
  if h : v ∈ B.inputs then c.states_equiv ⟨v, h⟩ (x (Sum.inl (c.ι ⟨v, h⟩).val))
  else x (Sum.inr ⟨v, h⟩)

variable (c) in
/-- On a variable that is not an input of 'B', 'restrictB' just reads the 'Sum.inr' component. -/
theorem restrictB_of_notMem (x : (composeNet c).Assignment) {v : B.V} (h : v ∉ B.inputs) :
  restrictB c x v = x (Sum.inr ⟨v, h⟩) := dif_neg h

variable (c) in
/-- Two kernel families side by side on the composite. -/
def composeKernel {R : Type} (κA : A.Kernel R) (κB : B.Kernel R) : (composeNet c).Kernel R
  | .inl m ⇒ fun x y ⇒ κA m (restrictA c x) y
  | .inr m ⇒ fun x y ⇒ κB m (restrictB c x) y

section Semantics

variable {R : Type}

/-- **Proposition 2 for sequential composition.** The joint of the composite is the product of
the two joints, each read off the corresponding part of the composite assignment. -/

```

```

theorem joint_compose [CommMonoid R] (κA : A.Kernel R) (κB : B.Kernel R)
  (x : (composeNet c).Assignment) :
  joint (composeKernel c κA κB) x = joint κA (restrictA c x) * joint κB (restrictB c x) := by
have h2 : ∀ m : B.M, composeKernel c κA κB (.inr m) x (x ((composeNet c).target (.inr m)))
  = κB m (restrictB c x) (restrictB c x (B.target m)) := fun m =>
  congrArg _ (restrictB_of_notMem c x (B.target_notMem_inputs m)).symm
unfold joint
rw [Fintype.prod_sum_type, Finset.prod_congr rfl fun m _ => h2 m]
rfl

theorem normalised_compose [AddCommMonoid R] [One R] (κA : A.Kernel R) (κB : B.Kernel R)
  (hA : ∀ m, Normalised κA m) (hB : ∀ m, Normalised κB m) :
  ∀ m, Normalised (composeKernel c κA κB) m
| .inl m => fun x => hA m (restrictA c x)
| .inr m => fun x => hB m (restrictB c x)

theorem local_compose (κA : A.Kernel R) (κB : B.Kernel R)
  (hA : ∀ m, Local κA m) (hB : ∀ m, Local κB m) :
  ∀ m, Local (composeKernel c κA κB) m
| .inl m => fun x x' hx => hA m (restrictA c x) (restrictA c x') fun p hp =>
  hx (Sum.inl p) (Finset.mem_map_of_mem _ hp)
| .inr m => fun x x' hx => hB m (restrictB c x) (restrictB c x') fun p hp => by
  have hxx := hx (c.tr p) (Finset.mem_map_of_mem c.trEmb hp)
  by_cases h : p ∈ B.inputs
  · rw [tr_of_mem h] at hxx
    simp only [restrictB, dif_pos h]
    exact congrArg _ hxx
  · rw [tr_of_notMem h] at hxx
    rw [restrictB_of_notMem c x h, restrictB_of_notMem c x' h]
    exact hxx

variable (c) in
/-- The composite variables contributed by 'A' (its own variables and the glued ones). -/
def varsA : Finset (composeNet c).V := Finset.univ.map Function.Embedding.inl

variable (c) in
/-- The composite variables contributed by 'B' alone: its non-input ("private") variables. -/
def varsB : Finset (composeNet c).V := Finset.univ.map Function.Embedding.inr

@[simp] theorem inl_mem_varsA (a : A.V) : (Sum.inl a : (composeNet c).V) ∈ varsA c :=
  Finset.mem_map_of_mem _ (Finset.mem_univ a)

@[simp] theorem inr_notMem_varsA (w : Priv B) : (Sum.inr w : (composeNet c).V) ∉ varsA c := by
  simp [varsA]

@[simp] theorem inr_mem_varsB (w : Priv B) : (Sum.inr w : (composeNet c).V) ∈ varsB c :=
  Finset.mem_map_of_mem _ (Finset.mem_univ w)

@[simp] theorem inl_notMem_varsB (a : A.V) : (Sum.inl a : (composeNet c).V) ∉ varsB c := by
  simp [varsB]

/-- The two halves are disjoint. -/
theorem disjoint_varsA_varsB : Disjoint (varsA c) (varsB c) := by
  rw [Finset.disjoint_left]
  rintro (a | w) hA hB
  · exact inl_notMem_varsB a hB
  · exact inr_notMem_varsA w hA

/-- The two halves of the composite's variables partition it. -/
theorem compl_varsA : (varsA c)c = varsB c := by
  ext v; cases v <|> simp [varsA, varsB]

/-- The composite mechanisms targeting the 'A' side are exactly 'A's. -/
theorem filter_target_mem_varsA :
  (Finset.univ.filter fun m => (composeNet c).target m ∈ varsA c)
  = Finset.univ.map Function.Embedding.inl := by

```

```

ext m; cases m <=> simp [varsA]

/-- **Proposition 3, downstream half, for open networks.** Summing the joint of the composite
over all values of 'B''s private variables leaves the joint of 'A' – the semantics of 'B ◦ A'
marginalised back onto 'A''s variables is the semantics of 'A'. Only 'B''s kernels have to be
normalised. -/
theorem marg_joint_compose [CommSemiring R] (κA : A.Kernel R) (κB : B.Kernel R)
  (hlocA : ∀ m, Local κA m) (hlocB : ∀ m, Local κB m) (hnormB : ∀ m, Normalised κB m)
  (x₀ : (composeNet c).Assignment) :
  marg (varsB c) (joint (composeKernel c κA κB)) x₀ = joint κA (restrictA c x₀) := by
have h := marg_joint_downstream (bn := composeNet c) (composeKernel c κA κB)
  (composeNet_target_injective c) (composeTopo c) (local_compose κA κB hlocA hlocB) (varsA c)
  (by
    rintro (m | m) hm
    · exact absurd (inl_mem_varsA (A.target m)) hm
    · exact fun x => hnormB m (restrictB c x))
  (by
    rintro (m | m) hm p hp
    · rw [composeNet_parents_inl, Finset.mem_map] at hp
      obtain ⟨a, _, rfl⟩ := hp
      exact inl_mem_varsA a
    · exact absurd hm (by simp [varsA]))
  (by
    rintro (a | w) hv
    · exact absurd (inl_mem_varsA a) hv
    · obtain ⟨m, hm⟩ := B.exists_mechanism w.property
      exact ⟨.inr m, congrArg Sum.inr (Subtype.ext hm)⟩)
x₀
rw [compl_varsA] at h
rw [h, filter_target_mem_varsA, Finset.prod_map]
rfl

/-- **Proposition 3 (sequential compositionality, SPEC §13.2, §61).** For any set 'S' of
composite variables coming from 'A' and any disjoint set 'T' coming from 'B''s private part,
integrating 'S ∪ T' out of the joint of the composite is the same as integrating 'T' out of
'B''s joint first, multiplying by 'A''s joint, and then integrating 'S' out. Taking 'S' to be
'A''s hidden variables together with the glued interface and 'T' to be 'B''s hidden variables,
this reads '[B ◦ A] = [B] ◦ [A]': the composite semantics is the sum over the interface of the
product of the two open semantics. -/
theorem marg_joint_compose_split [CommSemiring R] (κA : A.Kernel R) (κB : B.Kernel R)
  {S T : Finset (composeNet c).V} (hS : S ⊆ varsA c) (hT : T ⊆ varsB c)
  (x : (composeNet c).Assignment) :
  marg (S ∪ T) (joint (composeKernel c κA κB)) x
  = marg S (fun y => joint κA (restrictA c y))
    * marg T (fun z => joint κB (restrictB c z)) y x := by
have hjc : joint (composeKernel c κA κB)
  = fun z => joint κA (restrictA c z) * joint κB (restrictB c z) :=
  funext fun z => joint_compose κA κB z
rw [marg_union_disjoint ((disjoint_varsA_varsB (c := c)).mono hS hT), hjc]
unfold marg
refine Finset.sum_congr rfl fun y _ => ?_
refine marg_mul_left fun z hz => ?_
have : restrictA c z = restrictA c y := by
  funext a
  exact mem_fibre.1 hz (Sum.inl a) fun hmem => inl_notMem_varsB a (hT hmem)
rw [this]

variable (c) in
/-- Composing a *closed* network 'A' (no inputs) with an open 'B' gives a closed network: every
composite variable still has exactly one mechanism. -/
theorem closed_compose (h : A.inputs = ∅) : (composeNet c).Closed := by
  refine ⟨composeNet_target_injective c, ?_⟩
  rintro (a | w)
  · obtain ⟨m, hm⟩ := A.exists_mechanism (show a ∉ A.inputs by simp [h])
    exact ⟨.inl m, congrArg Sum.inl hm⟩
  · obtain ⟨m, hm⟩ := B.exists_mechanism w.property

```

```

    exact ⟨.inr m, congrArg Sum.inr (Subtype.ext hm)⟩

/-- Proposition 1a for a composite with a closed left factor: '[B ∘ A]' is a probability
distribution on the composite variables. -/
theorem sum_joint_compose_eq_one [CommSemiring R] (h : A.inputs = ∅) (κA : A.Kernel R)
  (κB : B.Kernel R) (hlocA : ∀ m, Local κA m) (hlocB : ∀ m, Local κB m)
  (hnormA : ∀ m, Normalised κA m) (hnormB : ∀ m, Normalised κB m) :
  ∑ x, joint (composeKernel c κA κB) x = 1 :=
  sum_joint_eq_one _ (closed_compose c h) (composeTopo c) (local_compose κA κB hlocA hlocB)
  (normalised_compose κA κB hnormA hnormB)

end Semantics

end Composable

end OpenFinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Markov.Basic

```

import Mathlib.CategoryTheory.MarkovCategory.Basic
import Mathlib.CategoryTheory.CopyDiscardCategory.Deterministic

```

Generic lemmas about Mathlib's abstract CopyDiscardCategory / MarkovCategory that the MarkovCategories.jl / BayesianNetworks.jl tests mirror for the concrete finite-stochastic instance (@instance ThMarkovCategory{FiniteSpace,FiniteKernel}):

- `discard_natural` — $f \gg \varepsilon[Y] = \varepsilon[X]$: composing a kernel with delete is delete. In Julia this is the test “discard naturality holds iff the kernel is normalised”; it is the axiom that ThMarkovCategory adds on top of ThMonoidalCategoryWithDiagonals.
- `deterministic_comp` — the composite of two deterministic (copy-preserving) kernels is deterministic; in Julia, point-mass kernels compose to point-mass kernels.
- `state_discard` — a state $p : I \rightarrow X$ composed with discard is the identity of the unit (a normalised distribution has total mass one).

No finite-stochastic MarkovCategory instance is attempted here (deferred; see the plan). Mathlib's Deterministic is IsComonHom, whose composition instance already exists; the theorem below simply names it.

```

namespace BayesianNetworksProofs.Markov

open CategoryTheory MonoidalCategory CopyDiscardCategory ComonObj

universe v u

variable {C : Type u} [Category.{v} C] [MonoidalCategory.{v} C]

section Markov

variable [MarkovCategory C]

/-- Discard is natural: every morphism of a Markov category is "normalised". -/
theorem discard_natural {X Y : C} (f : X → Y) : f >> ε[Y] = ε[X] :=
  MarkovCategory.discard_natural f

/-- A state has total mass one: composing 'p : I → X' with discard is the identity on 'I'. -/
theorem state_discard {X : C} (p : 1_C → X) : p >> ε[X] = 1 (1_C) := by
  rw [MarkovCategory.discard_natural, discard_unit]

end Markov

section CopyDiscard

```

```

variable [CopyDiscardCategory C]

/-- The composite of two deterministic morphisms is deterministic (Mathlib's 'IsComonHom'
composition instance, named here for the Julia test 'compose(δ_a, δ_b)' is a point mass). -/
theorem deterministic_comp {X Y Z : C} (f : X → Y) (g : Y → Z) [Deterministic f]
  [Deterministic g] : Deterministic (f >> g) :=
  inferInstance

/-- Identities are deterministic. -/
theorem deterministic_id (X : C) : Deterministic (1 X) :=
  inferInstance

/-- Deterministic morphisms commute with copy: 'f >> Δ = Δ >> (f ⊗ f)'. -/
theorem deterministic_copy {X Y : C} (f : X → Y) [Deterministic f] :
  f >> Δ[Y] = Δ[X] >> (f ⊗ f) :=
  Deterministic.copy_natural f

/-- The comonoid laws every object satisfies (the Julia comonoid-law tests). -/
example (X : C) : Δ[X] >> (ε[X] ▷ X) = (λ_ X).inv := counit_comul X
example (X : C) : Δ[X] >> (X ◁ ε[X]) = (ρ_ X).inv := comul_counit X
example (X : C) : Δ[X] >> (X ◁ Δ[X]) = Δ[X] >> (Δ[X] ▷ X) >> (α_ X X X).hom := comul_assoc X

end CopyDiscard

end BayesianNetworksProofs.Markov

```

BayesianNetworksProofs.Finite.Probability

```

import BayesianNetworksProofs.Finite.Intervention
import BayesianNetworksProofs.Finite.Tensor
import Mathlib.Data.NNReal.Defs

```

The finite model instantiated at $\mathbb{R}_{\geq 0}$, the value semiring of `FiniteKernel{Float64}` tables in `MarkovCategories.jl` (non-negative weights). Everything in `Finite/` is proved for an arbitrary commutative semiring; these corollaries just fix $R := \mathbb{R}_{\geq 0}$ so that the statements read as statements about probability distributions.

```

namespace BayesianNetworksProofs

namespace FinBayesNet

open scoped NNReal

/-- A family of non-negative real conditional kernels. -/
abbrev ProbKernel (bn : FinBayesNet) := bn.Kernel ℝ≥0

variable {bn : FinBayesNet}

/-- Proposition 1a over 'ℝ≥0': the joint of a closed, acyclic network with normalised local
kernels is a probability distribution on assignments. -/
theorem sum_joint_eq_one_nnreal (κ : bn.ProbKernel) (hclosed : bn.Closed) (ord : bn.TopoOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) : ∑ x, joint κ x = 1 :=
  sum_joint_eq_one κ hclosed ord hloc hnorm

/-- Proposition 4 over 'ℝ≥0': 'do(target m₀ = a)' truncates the factorisation. -/
theorem joint_intervene_nnreal (κ : bn.ProbKernel) (m₀ : bn.M) (a : bn.states (bn.target m₀))
  (x : bn.Assignment) :
  joint (intervene κ m₀ a) x =
    (if x (bn.target m₀) = a then 1 else 0) *
    ∏ m ∈ Finset.univ.erase m₀, κ m x (x (bn.target m)) :=
  joint_intervene κ m₀ a x

end FinBayesNet

```

```
end BayesianNetworksProofs
```

Roadmap (contains sorry)

```
import BayesianNetworksProofs.Finite.Open
```

Module `BayesianNetworksProofs.Roadmap`. Statements that are **not yet proved**. This module is deliberately *not* imported by the default target (`BayesianNetworksProofs.lean`) and is excluded from `Audit.lean`; build it with `lake build BayesianNetworksProofs.Roadmap` (or `make roadmap`). Every sorry here is listed in `README.md`.

Proposition 3 — the interface form

Proposition 3 itself, $\llbracket B \circ A \rrbracket = \llbracket B \rrbracket \circ \llbracket A \rrbracket$, is proved in `Finite/Open.lean` (`marg_joint_compose_split`) in the form

```
marg (S ∪ T)  $\llbracket B \circ A \rrbracket$  = marg S ( $\llbracket A \rrbracket \cdot$  marg T  $\llbracket B \rrbracket$ )
```

for any set S of composite variables coming from A and any disjoint set T coming from B 's private part, both marginals being taken *inside the composite*, over composite assignments. What remains is purely book-keeping, in two steps.

1. **Transfer.** `marg (S.map Sum.inl) (fun z  $\Rightarrow$  F (restrictA c z)) x = marg S F (restrictA c x)` and its B analogue. Both are `Finset.sum_nbij'` arguments: restricting a composite assignment is a bijection from the fibre of `S.map Sum.inl` onto the fibre of `S`, with inverse “keep the other component of x ”. Nothing deep, but a page of dependent-function extensionality.
2. **Interface sets.** Identifying the composite's hidden variables with $(A.\text{hidden} \cup \text{glued}) \oplus B.\text{hidden}$. This needs side conditions — no pass-through variables (`inputs  $\cap$  outputs =  $\emptyset$` on both sides) and a total interface match (`! surjective`, Julia's strict compose rather than the partial glue) — under which $(\text{compose } c).\text{hidden} = (A.\text{inputs}^c).\text{map Sum.inl} \cup ((B.\text{outputs}^c) \cap \text{Priv}).\text{map Sum.inr}$.

Together they turn `marg_joint_compose_split` into the statement below, which is the exact finite-model reading of SPEC §13.2 and §55.5: the semantics of the composite is the sum over the glued interface of the product of the two open semantics.

```
namespace BayesianNetworksProofs

namespace OpenFinBayesNet

namespace Composable

open FinBayesNet

variable {A B : OpenFinBayesNet} {R : Type} [CommSemiring R]

/-- **Proposition 3, interface form (unproved).** For a total interface match between two open
networks without pass-through variables, ' $\llbracket B \circ A \rrbracket = \llbracket B \rrbracket \circ \llbracket A \rrbracket$ ': the open semantics of the
composite is the sum, over all values of the glued interface variables, of the product of the
two open semantics. -/
theorem osem_compose (c : Composable A B) (κA : A.Kernel R) (κB : B.Kernel R)
  (hlocA : ∀ m, Local κA m) (hlocB : ∀ m, Local κB m) (hnormB : ∀ m, Normalised κB m)
  (hA : A.inputs  $\cap$  A.outputs =  $\emptyset$ ) (hB : B.inputs  $\cap$  B.outputs =  $\emptyset$ )
  (h1 : Function.Surjective c.1) (x : (compose c).Assignment) :
  osem (compose c) (composeKernel c κA κB) x =
    marg (c.glued.map Function.Embedding.inl)
      (fun y  $\Rightarrow$  osem A κA (restrictA c y) * osem B κB (restrictB c y)) x := by
  sorry

end Composable

end OpenFinBayesNet

end BayesianNetworksProofs
```

